PROJECT REPORT

AI-Powered Real-Time Live Transcription & Speaker Diarization

A locally hosted, GPU-accelerated monitoring platform for YouTube live streams and US Senate proceedings.

Prepared for

US Client — Controlled Pilot Evaluation

Prepared by

Southern Senate Project Team

Version 1.1 | July 23, 2026

Document Control

Project identity, status, audience, and purpose

Field	Details
Document title	AI-Powered Real-Time Live Transcription & Speaker Diarization
Version	1.1
Status	Functional prototype — ready for controlled pilot testing
Primary audience	Executive sponsors, product owners, technical reviewers, and pilot operators
Target use case	US Senate hearings, speeches, committee sessions, interviews, and public proceedings
Deployment model	Local Windows workstation with NVIDIA GPU; browser dashboard bound to localhost
Classification	Client project report; operational details should be handled according to client policy

Purpose of this report

This report documents the business need, implemented system, end-to-end workflow, technical architecture, operational safeguards, validation evidence, limitations, and recommended roadmap. It is intended to support a client demonstration and a structured decision on controlled pilot deployment.

Executive Summary

A near-live monitoring platform designed for accountable public proceedings

CURRENT STATE

Pilot-ready prototype

MODELS

Voxtral Mini 3B + 4B

VALIDATION

35 tests passed

The project is a locally hosted, GPU-accelerated system that monitors a live YouTube broadcast, continuously captures its audio, produces a near-live transcript, identifies speaker turns, assigns stable anonymous speaker labels, and presents video, text, colors, console output, and operational telemetry in one browser dashboard.

Business value

- Accelerates monitoring and review of Senate hearings, floor speeches, confirmation hearings, briefings, and interviews.
- Keeps inference local, reducing reliance on paid cloud transcription APIs and improving operational control over media processing.
- Preserves queued audio through a disk-backed first-in, first-out spool when processing temporarily falls behind.
- Makes different voices easier to follow through stable labels, consistent colors, and retrospective correction of delayed diarization decisions.
- Gives operators real-time visibility into source health, transcript backlog, real-time factor, CPU, RAM, and GPU memory.

Positioning

The current build is a functional prototype ready for controlled pilot testing. It is not an official record-generation platform and does not replace certified transcripts, stenographers, or human review. The design balances low delay, transcription completeness, and speaker accuracy; none of these can be treated as absolute under variable live-stream conditions.

Business Need, Objectives & Scope

The need

Public proceedings often contain hours of fast-moving testimony, questioning, and prepared remarks. Manual monitoring is expensive, difficult to search, and vulnerable to missed details. A near-live transcript with visible speaker turns provides a faster operational view while preserving the source video for context.

Project objectives

- Transcribe continuous live speech with minimal practical delay and without intentionally discarding queued content.
- Detect speaker changes and maintain stable Speaker 1, Speaker 2, and subsequent identities across a session.
- Recognize a returning questioner in an A → B → A exchange instead of creating a false third identity.
- Synchronize authenticated video and transcription against the same source family.
- Provide an operator-friendly dashboard, export capability, console visibility, and resource telemetry.
- Support safe model loading, switching, and unloading on a 12 GB-class GPU.

In scope	Out of scope for current prototype
YouTube live acquisition with browser-cookie authentication	Certified or legally authoritative transcripts
Local Voxtral transcription and Pyannote diarization	Automatic verified naming of senators and witnesses
Anonymous stable speaker labels and colors	Perfect word-level speaker boundaries
Authenticated video preview and transcript export	Public internet deployment and multi-tenant access
Operational telemetry and crash safeguards	Formal WER/DER benchmark results

System Architecture

Modular local processing with separate media, AI, reliability, and presentation layers

Presentation Layer

Browser dashboard • authenticated video • transcript • controls • telemetry

Application & API Layer

Flask routes • session lifecycle • model lifecycle • Server-Sent Events

Media Ingestion Layer

yt-dlp • Firefox cookies • Deno/EJS • FFmpeg audio/video pipelines

AI Inference Layer

Voxtral transcription • Pyannote diarization • voice embeddings • identity reconciliation

Reliability Layer

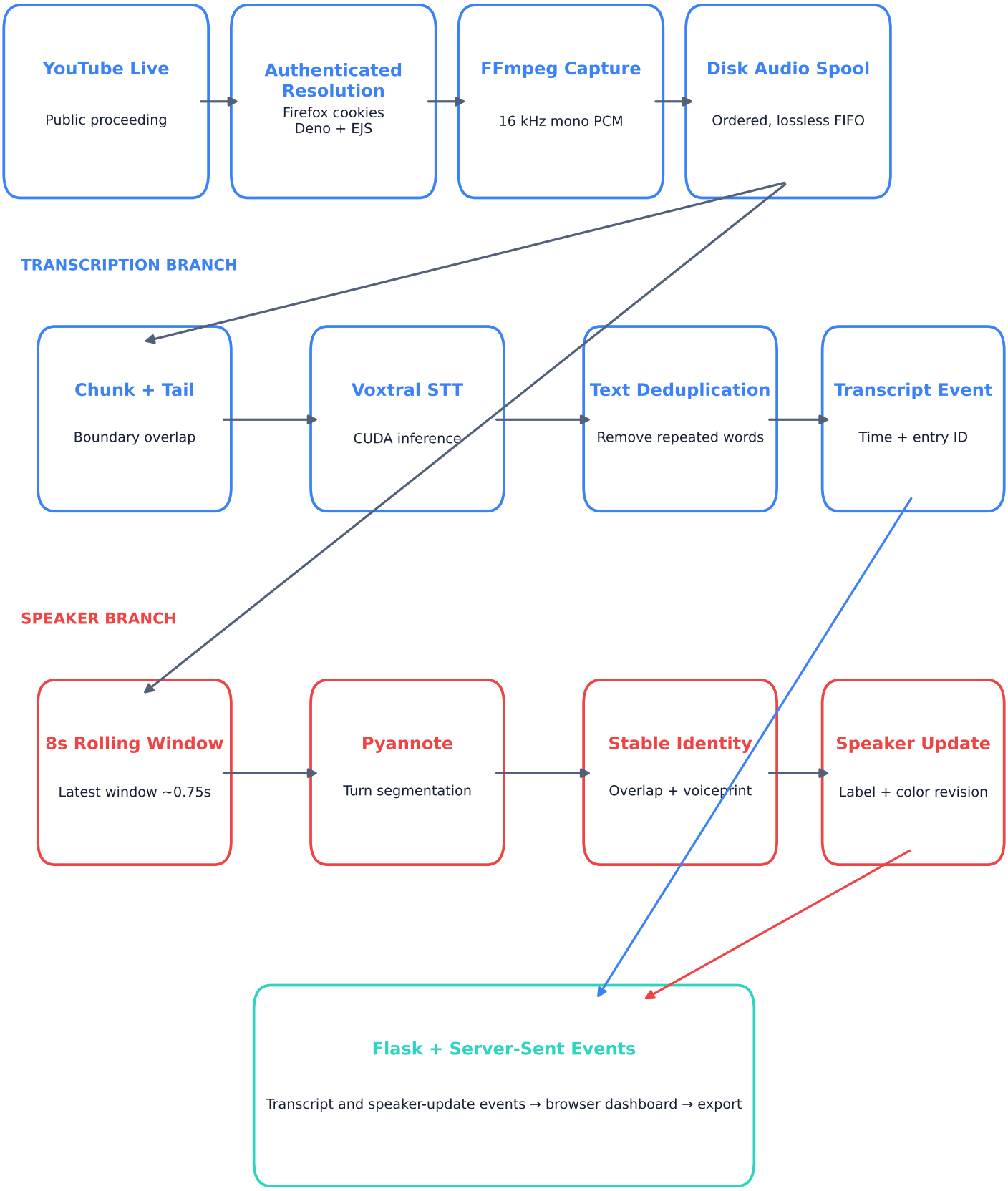
Disk spool • retries • backlog metrics • duplicate-instance guard • CUDA cleanup

Hardware Layer

Windows • Python 3.11 • RTX 5070 • CUDA • local RAM and disk

End-to-End Workflow

Audio continuity and asynchronous speaker analysis converge in the live dashboard



Implemented Feature Matrix

Capability	Current implementation	Status
Live acquisition	Browser-cookie authentication, Deno/EJS challenge solving, authenticated audio and video resolution	Implemented
Speech-to-text	Local Voxtral Mini 3B/4B selection, deterministic decoding, configurable chunk/tail/delay controls	Implemented
Content continuity	Disk-backed FIFO spool, ordered processing, acknowledgement after successful inference	Implemented
Speaker diarization	Latest-only overlapping Pyannote analysis with up to ten stable session speakers	Implemented
Stable identity	Time-overlap reconciliation and asynchronous voiceprint matching for returning speakers	Implemented
Visual speaker tracking	Speaker labels, stable colors, colored transcript borders, retrospective row correction	Implemented
Operator dashboard	Authenticated video, live transcript, console, controls, status indicators, clear and export	Implemented
Telemetry	CPU, RAM, GPU, VRAM, words, chunks, source state, backlog, and real-time factor	Implemented
Model lifecycle	Safe load, switch, unload, CUDA cleanup, duplicate request coalescing	Implemented
Named public figures	Automatic mapping of anonymous voices to verified senator/witness names	Proposed
Formal benchmark	WER, DER, change-delay, and end-to-end latency evaluation on Senate material	Pending

Speech-to-Text Pipeline

Near-live processing designed to preserve audio continuity

- FFmpeg decodes the resolved live stream to 16 kHz, mono, signed 16-bit PCM audio.
- Audio is divided into configurable chunks; the verified live profile normally uses approximately two-second chunks.
- A short tail, normally 0.3 seconds, is prepended to the next chunk to reduce words being clipped at chunk boundaries.
- Text-level overlap removal eliminates repeated words introduced by the audio tail.
- Temperature defaults to 0.0 and output tokens are limited to reduce hallucination, rambling, and unnecessary latency.
- Silence-threshold processing avoids spending GPU inference time on sufficiently quiet chunks.
- Each chunk is written to a disk-backed spool and acknowledged only after processing, protecting continuity during transient inference slowdown.
- Each transcript decision remains tied to a bounded audio interval; oversized ten-second catch-up batches were removed because they obscured speaker changes.

Operational latency model

Observed end-to-end timing is the sum of upstream YouTube broadcast delay, stream acquisition, chunk duration, model inference, browser delivery, and any backlog. The dashboard therefore reports real-time factor and backlog rather than claiming impossible zero latency. When the real-time factor remains below 1.0, inference is faster than incoming audio.

SAMPLE RATE

16 kHz mono

DEFAULT TAIL

0.3 seconds

DECODING

Temperature 0.0

Speaker Diarization & Identity Management

Concept	Role
Speaker-change detection	Determines that the active voice has changed.
Diarization	Segments audio by anonymous speaker turns.
Stable identity	Maps window-local labels to persistent session identities.
Speaker labeling	Displays Speaker 1, Speaker 2, and subsequent anonymous labels.
Color assignment	Maintains a consistent visual color for each stable identity.

Implemented method

- Pyannote analyzes the newest overlapping eight-second rolling window approximately every 0.75 seconds once enough speech is available.
- Window-local speaker labels are reconciled against the previous window using absolute timeline overlap.
- Voice embeddings are calculated asynchronously and compared with session profiles to support A → B → A identity reuse.
- The live registry supports up to ten stable speaker identities and reuses returning voice identities instead of inventing a new speaker for each turn.
- Transcription does not wait for diarization. Recent transcript rows retain entry IDs and audio positions.
- When diarization finishes later, a speaker-update event revises the affected row’s label, color, and border.

Important limitation

Reliable voice identification requires a short evidence window. Rapid interruptions, overlapping speech, similar voices, broadcast mixing, and poor microphones remain difficult. Without word-level alignment, a speaker change inside one transcription chunk cannot always be placed at the exact word boundary.

Authenticated YouTube Integration & Operator Expe

Authentication design

YouTube may require “Sign in to confirm you’re not a bot.” The application does not request the operator’s username or password. Instead, yt-dlp reads cookies from an explicitly selected browser profile. Firefox was verified locally because Chrome and Edge can lock or encrypt their cookie databases through Windows protections.

- yt-dlp 2026.07.04 performs media extraction.
- yt-dlp-ejs and Deno 2.9.3 solve current YouTube JavaScript challenges.
- The video preview uses an authenticated server-side proxy instead of an iframe with a separate cookie context.
- Low-buffer FFmpeg flags, smaller proxy writes, and no-cache response headers reduce unnecessary preview drift.
- The preview displays CONNECTING, LIVE, or DELAYED status and offers a Jump to live control when browser playback falls behind.

Dashboard controls and feedback

Area	Functions
Session	URL input, Start/Stop, source state, authenticated preview
Transcript	Live rows, labels, colors, clear, TXT export
Model	Select, load, unload, safe switching
Tuning	Language, chunk, RMS, tokens, temperature, overlap, delay
Operations	Console, CPU/RAM/GPU, backlog, RTF, words, chunks

Technology Stack

Technology	Role	Rationale
Python 3.11	Backend and AI orchestration	Mature ecosystem and direct integration with model tooling
Flask + SSE	Local API and live events	Simple threaded local server and low-overhead one-way updates
HTML/CSS/JavaScript	Operator dashboard	Portable browser experience without a heavy frontend build
PyTorch + CUDA	GPU inference	Local acceleration on NVIDIA RTX hardware
Transformers	Voxtral loading and inference	Model configuration and generation runtime
Mistral Common	Audio protocol	Voxtral streaming/transcription request structures
Voxtral Mini Realtime	Speech-to-text	Open-weights, local near-live transcription
Pyannote Audio	Speaker diarization	Speaker-turn segmentation and embedding support
FFmpeg 8.1.2	Media processing	Resampling, decoding, reconnection, and video proxying
yt-dlp + EJS	YouTube extraction	Cookie authentication and current challenge support
Deno 2.9.3	JavaScript runtime	Recommended runtime for YouTube challenge solving
NumPy / psutil	Signal and telemetry	Embedding math and system resource reporting
PowerShell	Windows launcher	Environment checks and duplicate-instance protection
unittest	Automated validation	Fast repeatable contract and behavior checks

Reliability, GPU Safety & Failure Recovery

Runtime safeguards

- Disk-backed FIFO buffering preserves order and retains unacknowledged chunks through transient inference failure.
- FFmpeg reconnect options handle recoverable streamed-media interruptions.
- Model downloads and loading use retry/backoff behavior for transient hosting failures.
- Diarization requires three consecutive failures before it is disabled; transcription remains independent.
- Voice embedding is treated as an enhancement and cannot terminate transcription.
- A minimum two-gigabyte disk reserve is required before the live spool proceeds.
- Source idle time, processing state, real-time factor, and backlog are exposed to the operator.

Memory-safe model lifecycle

- Only one transcription model is intentionally held in memory.
- Switching clears model and processor references, runs garbage collection, empties CUDA cache, and attempts CUDA IPC cleanup.
- Duplicate model-load requests are coalesced to prevent accidental double loading.
- Unload stops active work first and reports freed/remaining PyTorch VRAM.
- The server checks port 7860 before background model loading. A duplicate app launch is rejected before CUDA allocation.
- The PowerShell launcher detects a healthy existing instance and exits safely.

LOADED MODEL

~8.25 GB allocated

AFTER UNLOAD

0.00 GB allocated

DUPLICATE LAUNCH

Rejected pre-CUDA

Security, Privacy & Data Governance

Current posture

- Speech recognition and speaker analysis run locally on the workstation.
- The application does not collect browser usernames or passwords.
- yt-dlp accesses cookies only from the operator-selected local browser profile.
- Resolved media URLs are temporary and are not displayed in application logs.
- Flask binds to 127.0.0.1, debug mode is disabled, and the dashboard is not publicly exposed by default.
- Transcript exports and runtime audio spools remain local and should be governed by an agreed retention policy.

Production controls recommended

Control area	Recommended production measure
Identity and access	Authenticated users, role-based permissions, session timeout
Transport	HTTPS/TLS and secure reverse proxy
Storage	Encryption at rest, protected export folders, managed retention
Auditability	Operator actions, corrections, exports, model/configuration history
Secrets	Managed handling of tokens and browser-auth alternatives
Network	Firewall policy, allowlisted sources, monitored outbound access
Governance	Privacy review, records policy, incident response, data classification

Testing & Validation Evidence

Final presentation-readiness checks completed on the target workstation

Check	Evidence	Result
Automated test suite	35 tests passed	PASS
Python compilation	Core backend, UI, diarization, and spool modules	PASS
Python dependency integrity	No broken requirements reported	PASS
PowerShell launcher	Parser completed with zero errors	PASS
Dashboard/API	/, /status, /models, and /metrics returned HTTP 200	PASS
YouTube authentication	Exact live audio and video URLs resolved with Firefox cookies	PASS
Media ingestion	Five-second FFmpeg audio and video decode smoke tests	PASS
Model runtime	Voxtral Mini 3B and 4B selectable with safe CUDA switching	PASS
Diarization runtime	Pyannote initialized and reported ready	PASS
Model lifecycle	Unload freed ~8.25 GB; reload completed successfully	PASS
Duplicate launch	Rejected before GPU model allocation	PASS
Disk capacity	~513 GB free on project drive during validation	PASS

Metrics not yet formally measured

Word error rate (WER), diarization error rate (DER), speaker-change delay, and 95th-percentile end-to-end latency remain pending a representative Senate benchmark dataset.

Risks & Mitigations

Risk	Impact	Mitigation	Residual
YouTube authentication changes	Stream resolution may fail	Keep yt-dlp/EJS/Deno current; support alternate official feeds	Medium
Network or source interruption	Temporary missing live input	FFmpeg reconnect, source-idle telemetry, disk spool	Medium
GPU out-of-memory	Process crash or failed model load	3B/4B selection, unload control, duplicate guards, one-model lifecycle	Low
Speaker overlap/cross-talk	Incorrect speaker boundaries	Retrospective updates, operator correction roadmap, human review	High
Similar voices	False identity merge or split	Voice profiles, overlap reconciliation, controlled benchmark tuning	Medium
Upstream live delay	Video/text timing mismatch	Authenticated proxy, same source family, live-edge status and jump control	Medium
Transcript hallucination/error	Incorrect record	Temperature 0, token bounds, source video, human verification	Medium
Local data retention	Privacy or records exposure	Formal retention, encryption, controlled exports	Medium
Single workstation failure	Loss of service	Production service supervision, health checks, redundant deployment	Medium

Recommended Roadmap for a US Senate Pilot

Phase 1 — Current Prototype

COMPLETED

- Local live transcription and authenticated video
- Anonymous stable speakers and colors
- Operational telemetry and export
- Crash and GPU lifecycle safeguards

Phase 2 — Pilot Validation

NEXT

- Build representative Senate evaluation set
- Measure WER, DER, latency, and turn-delay KPIs
- Run multi-hour endurance and interruption tests
- Collect operator usability feedback

Phase 3 — Senate Enhancement

PLANNED

- Committee rosters and hearing metadata
- Operator rename/merge/split controls
- Word-level alignment and citation timestamps
- Entities, bills, topics, alerts, and Q&A grouping

Phase 4 — Production Hardening

FUTURE

- Authentication, RBAC, HTTPS, and audit trails
- Service supervision and centralized monitoring
- Retention, encryption, and records governance
- Redundancy and approved deployment architecture

Pilot Success Metrics

KPI	Definition	Current state
Transcript latency	Median and 95th percentile from source audio to displayed text	Pending benchmark
Transcription quality	Word error rate by hearing type, speaker, and audio condition	Pending benchmark
Diarization quality	Diarization error rate and speaker confusion rate	Pending benchmark
Turn responsiveness	Median speaker-change detection and correction delay	Pending benchmark
Identity stability	A → B → A returning-speaker accuracy	Pending benchmark
Continuity	Percentage of captured audio processed; dropped-audio seconds	Architecture targets zero intentional drops
Throughput	Average/maximum backlog and real-time factor	Available in dashboard
Reliability	Session uptime and recovery time after source interruption	Pending endurance test
Resource stability	Peak VRAM/RAM and absence of growth over multi-hour runs	Pending endurance test
Operator quality	Correction rate, task completion time, and usability score	Pending pilot feedback

Recommended acceptance approach

Agree target thresholds with the client before pilot scoring. Benchmark against manually reviewed Senate material covering prepared remarks, rapid questioning, interruptions, remote witnesses, accents, poor microphones, and overlapping speech. Report aggregate results and failure-case slices.

Limitations & Conclusion

Known limitations

- Zero latency is physically impossible; the product is near-live and inherits YouTube’s upstream broadcast delay.
- YouTube authentication and anti-bot mechanisms can change without notice.
- Cross-talk, interruptions, similar voices, noise, and broadcast mixing reduce diarization accuracy.
- Anonymous speaker numbers are not verified personal identities.
- Exact word-level speaker boundaries require an additional alignment stage.
- A 12 GB-class GPU limits safe model size and available concurrency.
- Official records, quotations, legal use, and publication require human review against the source.

Conclusion

The project has reached a strong prototype milestone: authenticated live media acquisition, local GPU transcription, continuous audio protection, stable anonymous speaker handling, retrospective color correction, operator telemetry, model lifecycle controls, and presentation-focused crash safeguards are integrated and locally validated.

The recommended next step is a controlled Senate pilot centered on measurable transcription quality, speaker accuracy, end-to-end delay, multi-hour stability, and operator correction workflows. With those results, the client can make an evidence-based decision on Senate-specific enhancement and production hardening.

CURRENT STATUS: FUNCTIONAL PROTOTYPE — READY FOR CONTROLLED PILOT TESTING